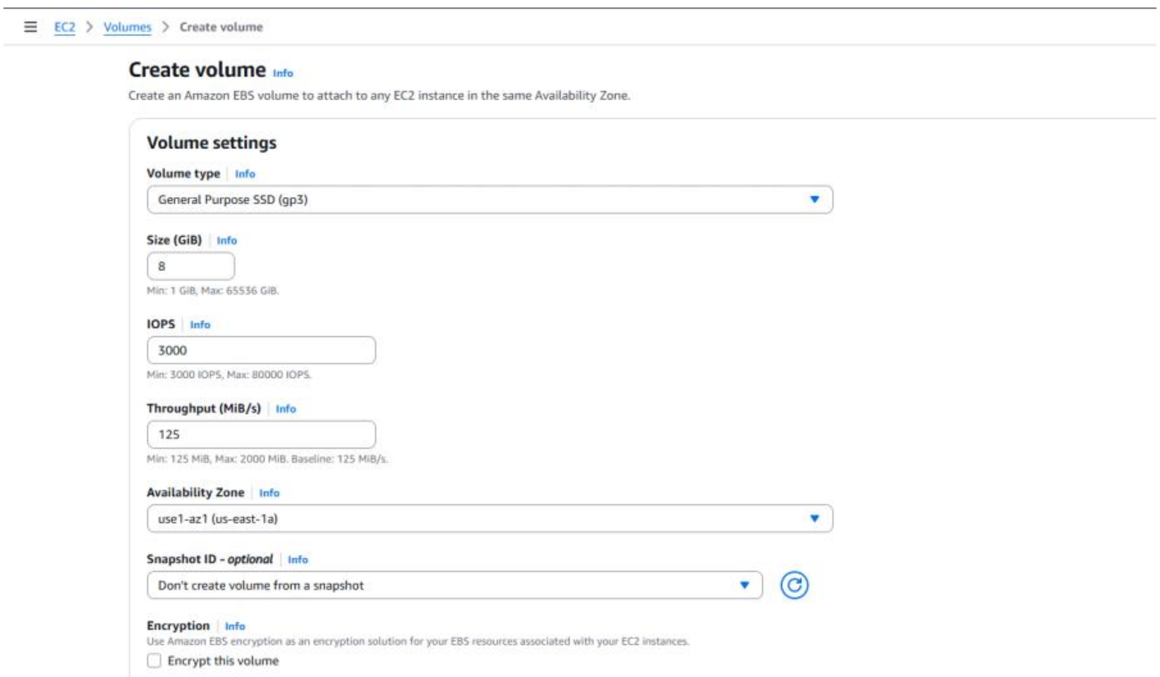
Automated EBS Snapshot Creation and Cleanup

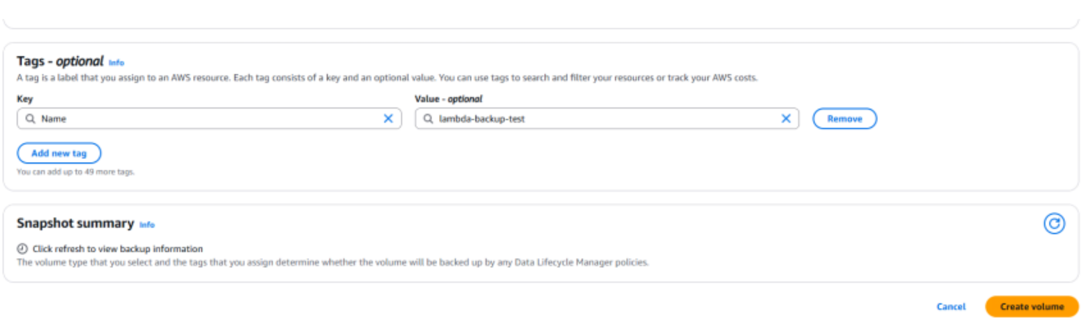
Monday, July 13, 2026 5:16 PM

1. EBS Volume Setup

Console Navigation

1. Sign in to the **AWS Management Console**.
2. In the top search bar, type **EC2** and select it.
3. In the left sidebar, under **Elastic Block Store**, click **Volumes**.
4. To use an existing volume: note the **Volume ID** (starts with vol-) from the list.
5. To create a new test volume:
   - Click **Create volume** (top right).
   - **Volume type**: gp3
   - **Size**: 8 GiB
   - **Availability Zone**: pick one matching your test EC2 instance's AZ (e.g., us-east-1a)
   - Scroll to **Tags** → **Add tag** → Key: Name, Value: lambda-backup-test
   - Click **Create volume**.
6. Copy the new **Volume ID** — you'll need it for the Lambda environment variable.





 # Create a new test volume
aws ec2 create-volume --availability-zone us-east-1a --size 8 --volume-type gp3 --tag-specifications "ResourceType=volume,Tags=[{Key=Name,Value=lambda-backup-test}]"

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task2\ebs-snapshot-cleanup> aws ec2 create-volume --availability-zone us-east-1a --size 8 --volume-type gp3 --tag-specifications "ResourceType=volume,Tags=[{Key=N
ame,Value=lambda-backup-test}]"
{
    "AvailabilityZoneId": "use1-az1",
    "Iops": 3000,
    "Tags": [
        {
            "Key": "Name",
            "Value": "lambda-backup-test"
        }
    ],
    "VolumeType": "gp3",
    "MultiAttachEnabled": false,
    "Throughput": 125,
    "VolumeId": "vol-03fb1ddae64ddfed9",
    "Size": 8,
    "SnapshotId": "",
    "AvailabilityZone": "us-east-1a",
    "State": "creating",
    "CreateTime": "2026-07-13T14:14:25+00:00",
    "Encrypted": false
}
```

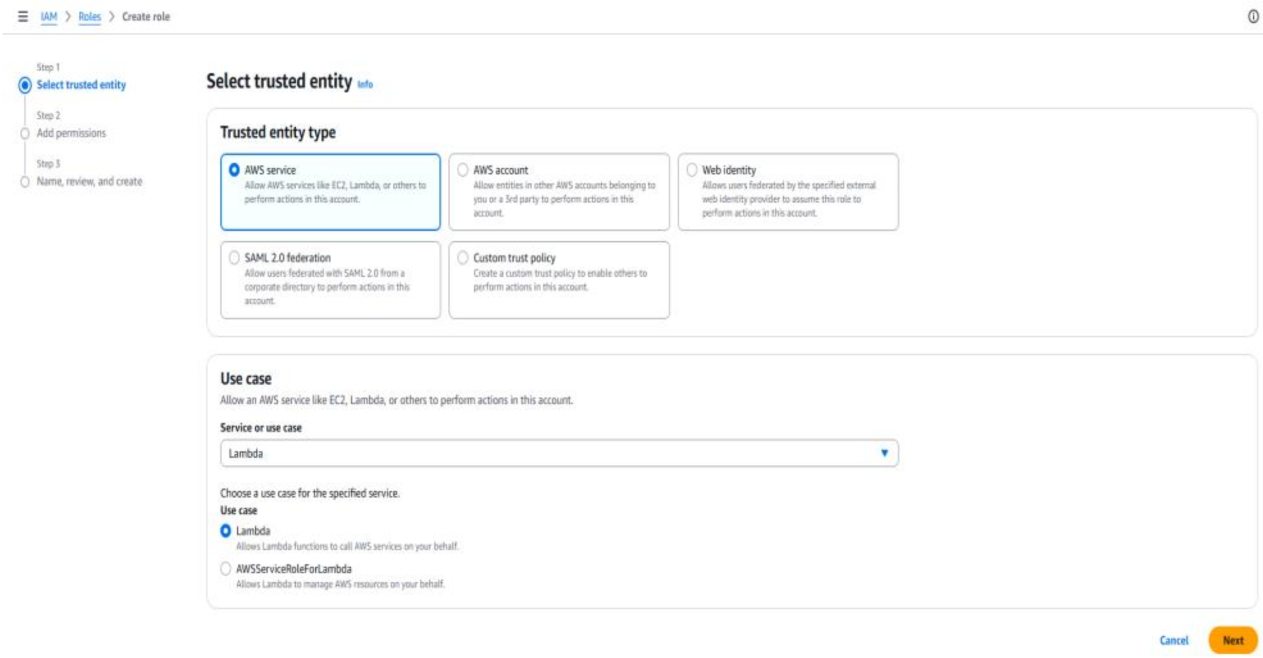
```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task2\ebs-snapshot-cleanup> aws ec2 describe-volumes
{
    "Volumes": [
        {
            "AvailabilityZoneId": "use1-az1",
            "Iops": 3000,
            "Tags": [
                {
                    "Key": "Name",
                    "Value": "lambda-backup-test"
                }
            ],
            "VolumeType": "gp3",
            "MultiAttachEnabled": false,
            "Throughput": 125,
            "Operator": {
                "Managed": false
            },
            "VolumeId": "vol-03fb1ddae64ddfed9",
            "Size": 8,
            "SnapshotId": "",
            "AvailabilityZone": "us-east-1a",
            "State": "available",
            "CreateTime": "2026-07-13T14:14:25.867000+00:00",
            "Attachments": [],
            "Encrypted": false
        }
    ]
}

PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task2\ebs-snapshot-cleanup>
```

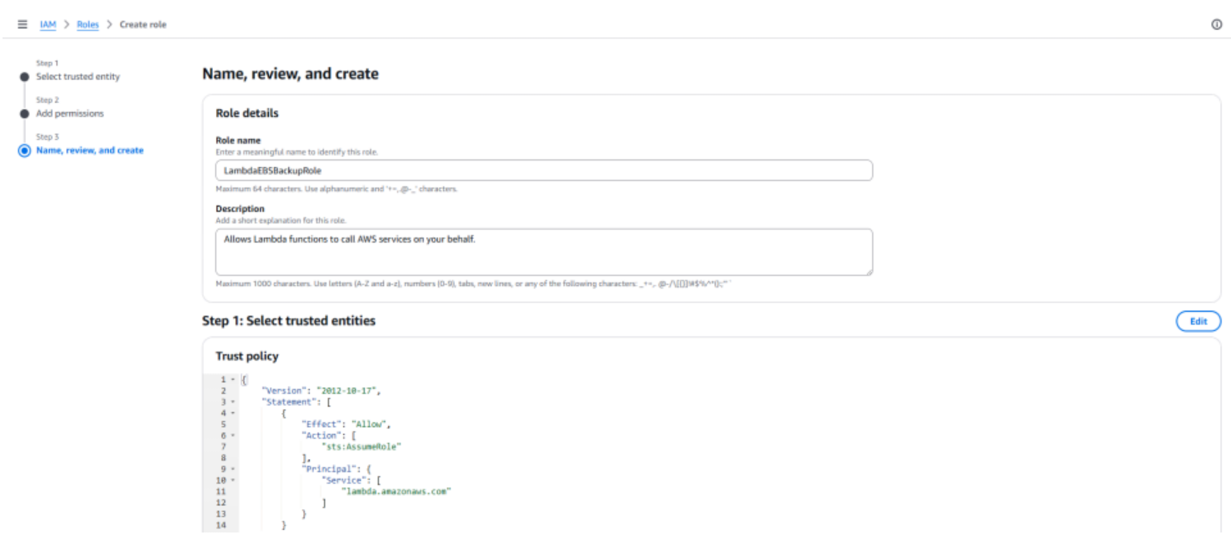
2. IAM Role for Lambda

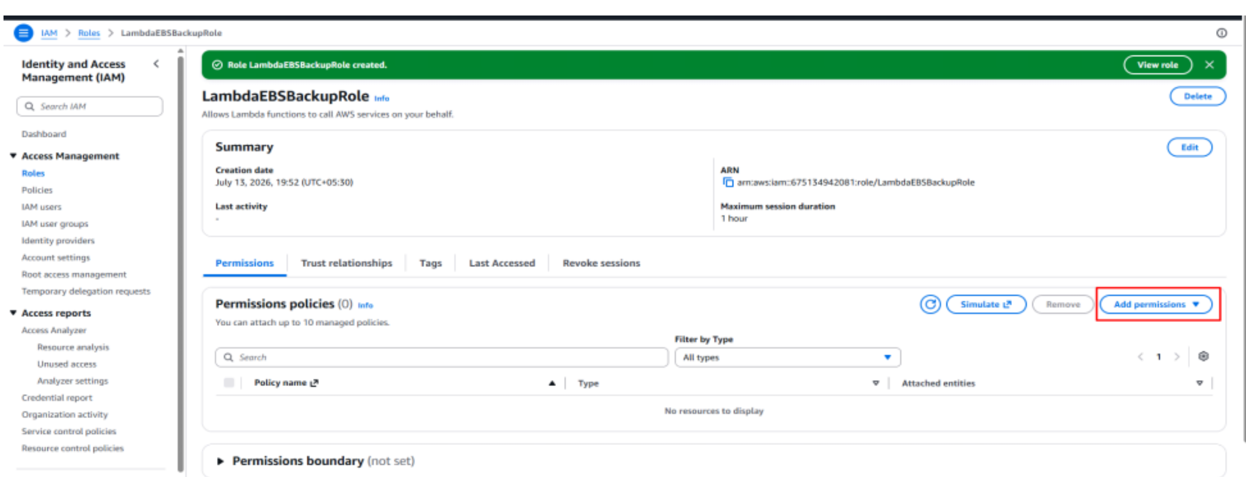
Console Navigation

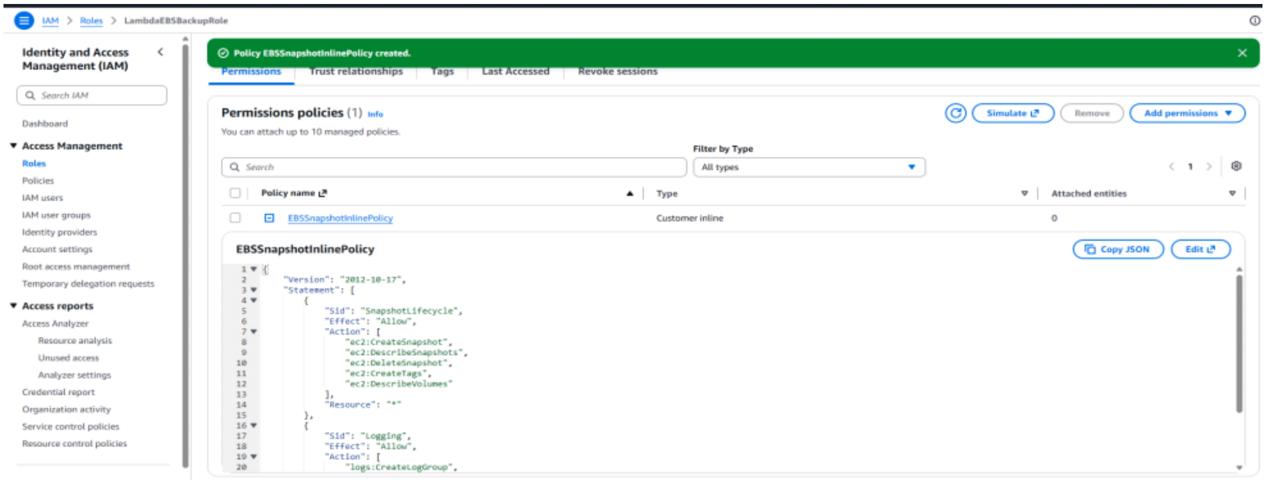
1. Search bar → **IAM** → open the IAM console.
2. Left sidebar → **Roles** → **Create role**.
3. **Trusted entity type**: AWS service
4. **Use case**: select Lambda → **Next**.
5. On the **Add permissions** page, skip attaching a managed policy for now (you'll add an inline one after creation) → **Next**.
6. **Role name**: LambdaEBSBackupRole → **Create role**.
7. Open the newly created role → **Add permissions** dropdown → **Create inline policy**.

8. Click the **JSON** tab and paste:





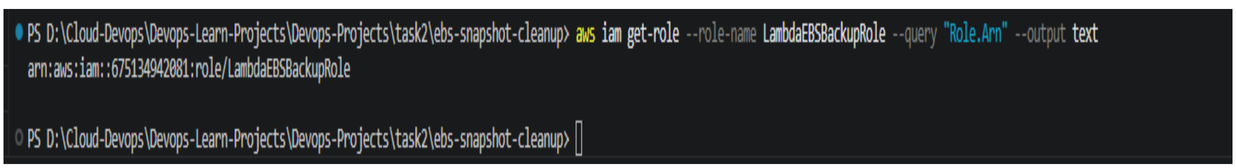




arn:aws:iam::675134942081:role/LambdaEBSBackupRole

 # Get the Role ARN

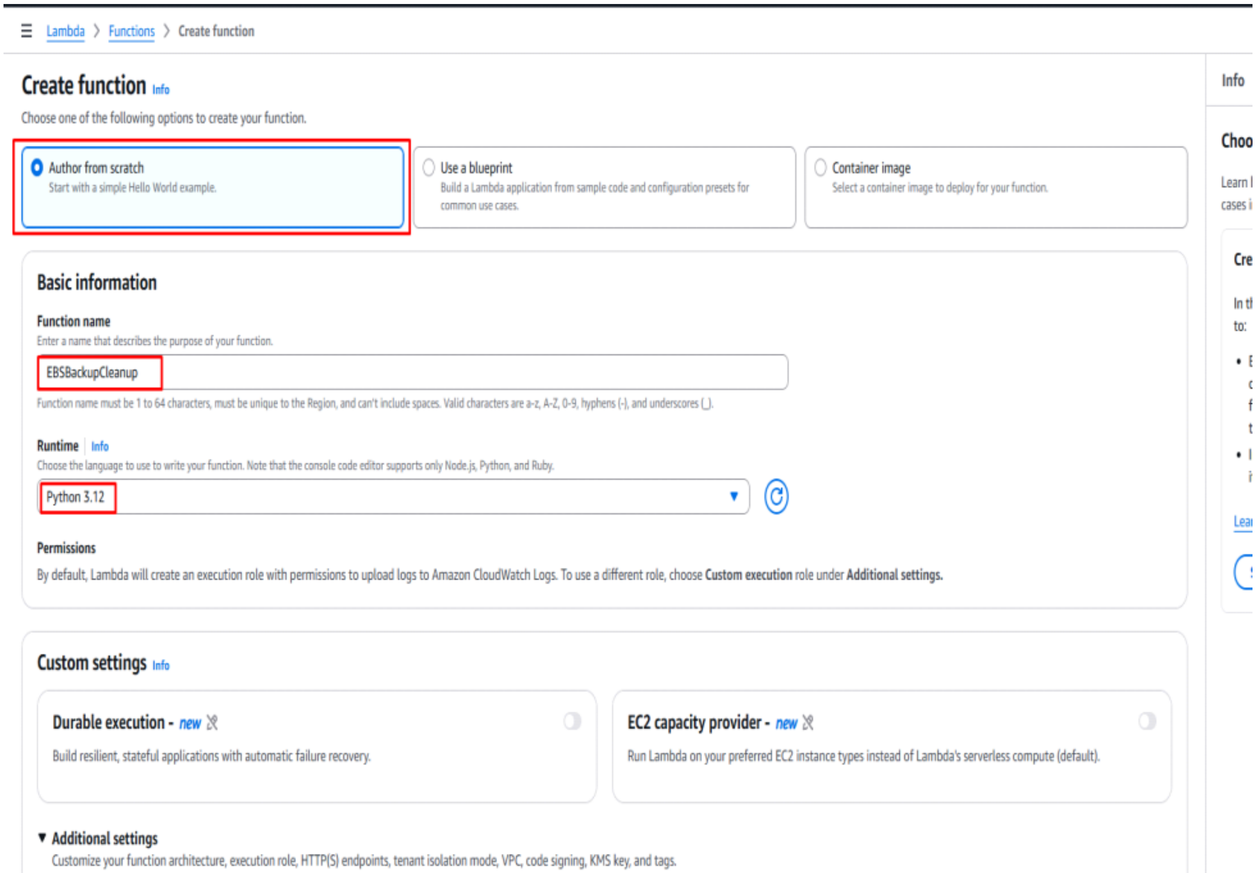
aws iam get-role --role-name LambdaEBSBackupRole --query "Role.Arn" --output text

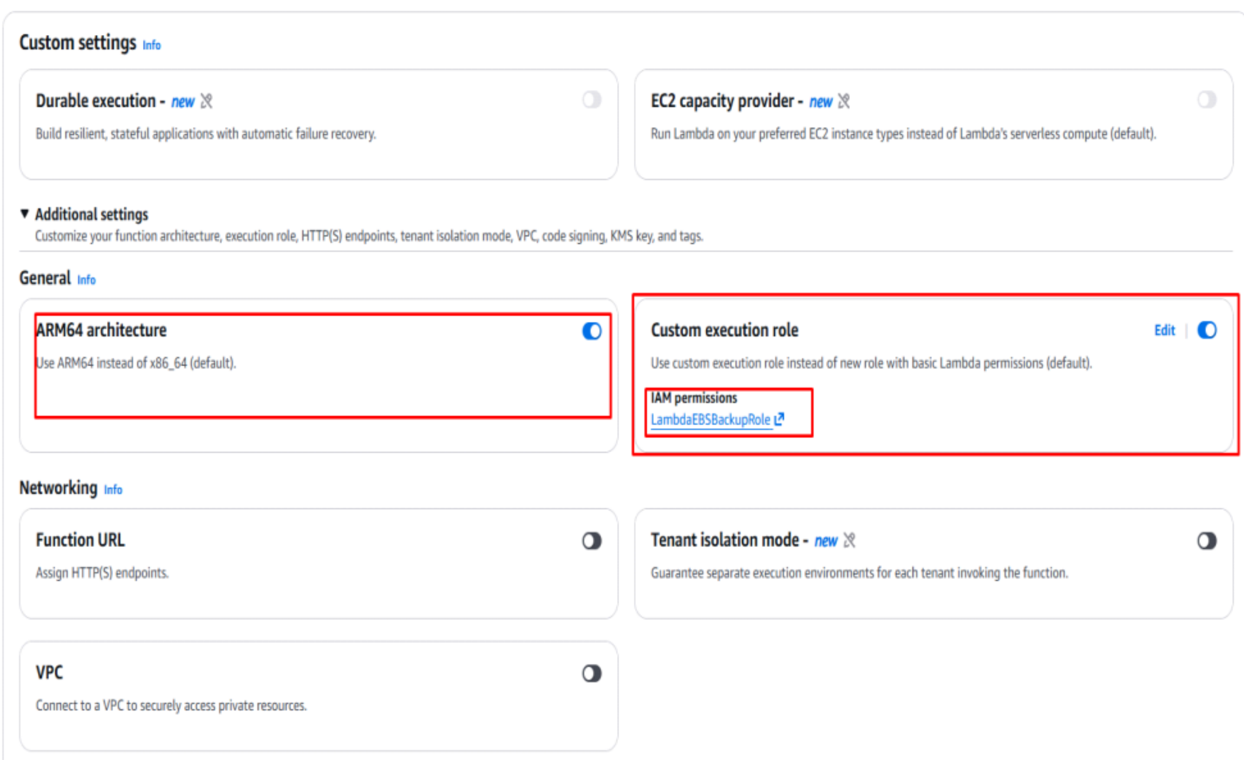


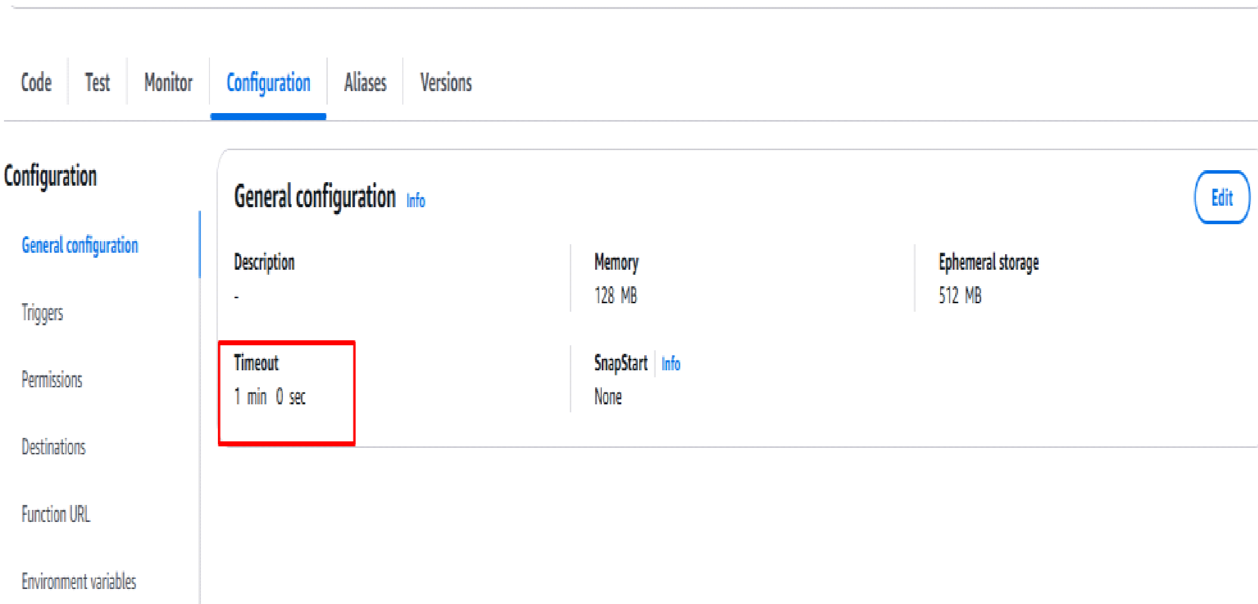
3. Create the Lambda Function

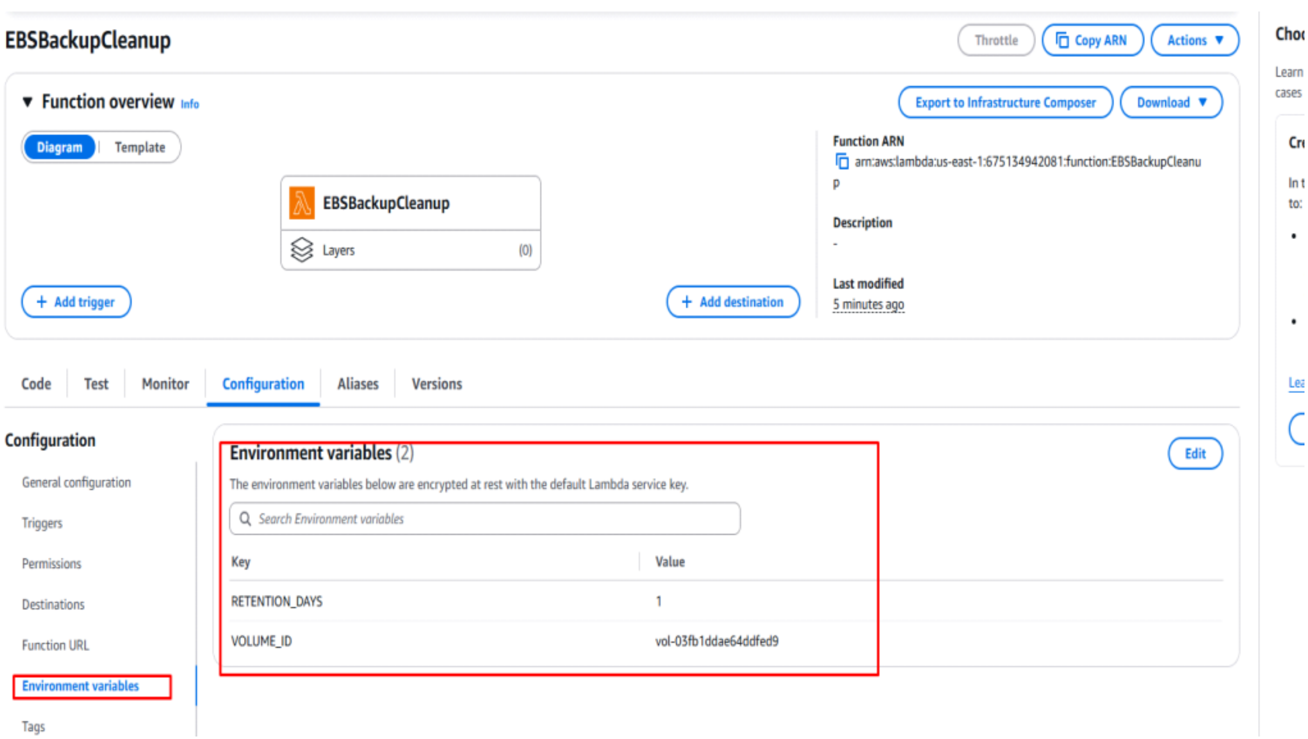
Console Navigation

1. Search bar → **Lambda** → open the Lambda console.
2. Click **Create function**.
3. Select **Author from scratch**.
4. **Function name**: EBSBackupCleanup
5. **Runtime**: Python 3.12
6. **Architecture**: x86_64
7. Expand **Change default execution role** → select **Use an existing role** → choose LambdaEBSBackupRole.
8. Click **Create function**.
9. In the **Code source** editor, delete the placeholder code and paste the script below (see full code in the next section).
10. Click **Deploy**.
11. Go to the **Configuration** tab → **General configuration** → **Edit**:
    - **Timeout**: set to 1 min 0 sec
    - Save.
12. Configuration tab → **Environment variables** → **Edit** → **Add environment variable**:
    - Key: VOLUME_ID, Value: vol-0123456789abcdef0 (your actual volume ID)
    - Key: RETENTION_DAYS, Value: 30
    - Save.


Lambda
Functions
Create function
Create function Info
Choose one of the following options to create your function.
Author from scratch
Start with a simple Hello World example.
Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.
Container image
Select a container image to deploy for your function.
Basic information
Function name
Enter a name that describes the purpose of your function.
EBSBackupCleanup
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).
Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.12
Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose Custom execution role under Additional settings.
Custom settings Info
Durable execution - new
Build resilient, stateful applications with automatic failure recovery.
EC2 capacity provider - new
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).
Additional settings
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.
Info


Custom settings Info
Durable execution - new
Build resilient, stateful applications with automatic failure recovery.
EC2 capacity provider - new
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).
Additional settings
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.
General Info
ARM64 architecture
Use ARM64 instead of x86_64 (default).
Custom execution role
Edit
Use custom execution role instead of new role with basic Lambda permissions (default).
IAM permissions
LambdaEBSBackupRole
Networking Info
Function URL
Assign HTTP(S) endpoints.
Tenant isolation mode - new
Guarantee separate execution environments for each tenant invoking the function.
VPC
Connect to a VPC to securely access private resources.



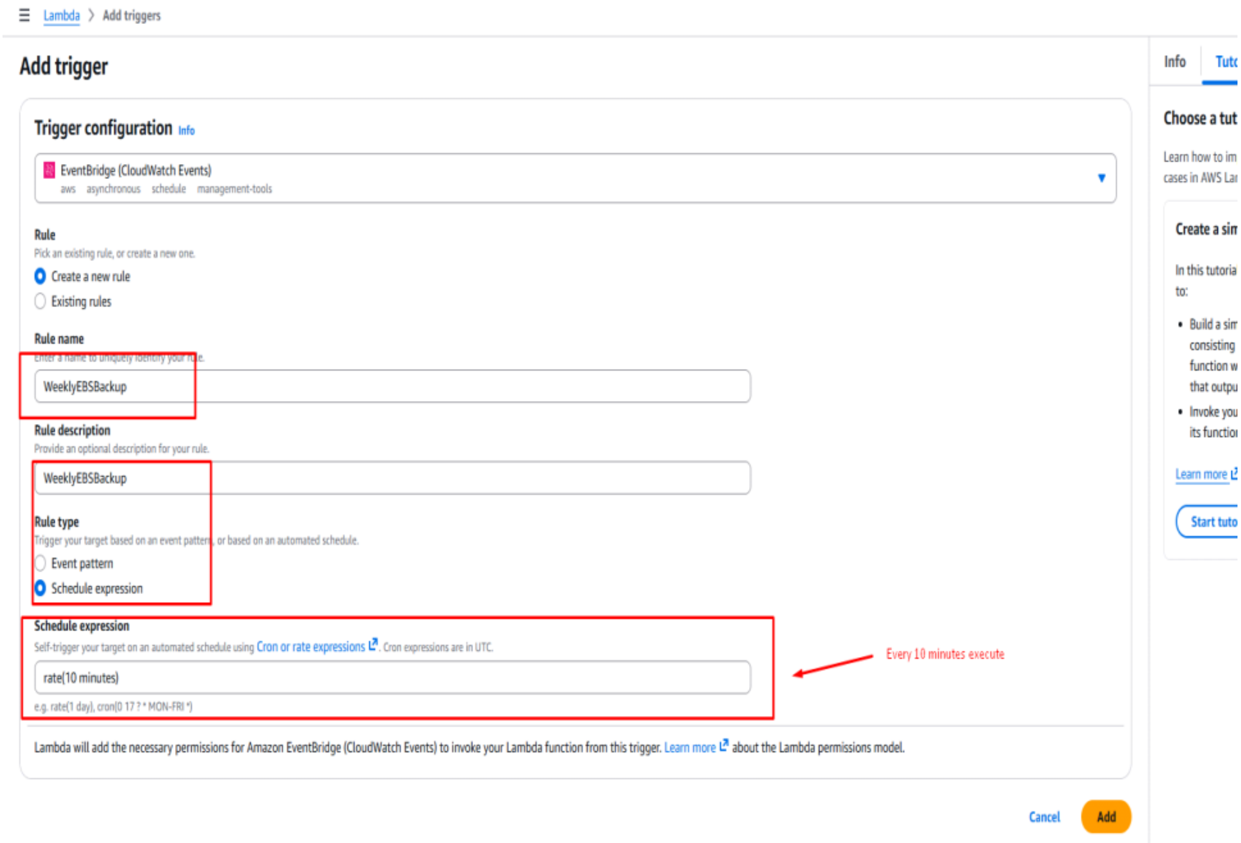


4. EventBridge Weekly Schedule

Console Navigation

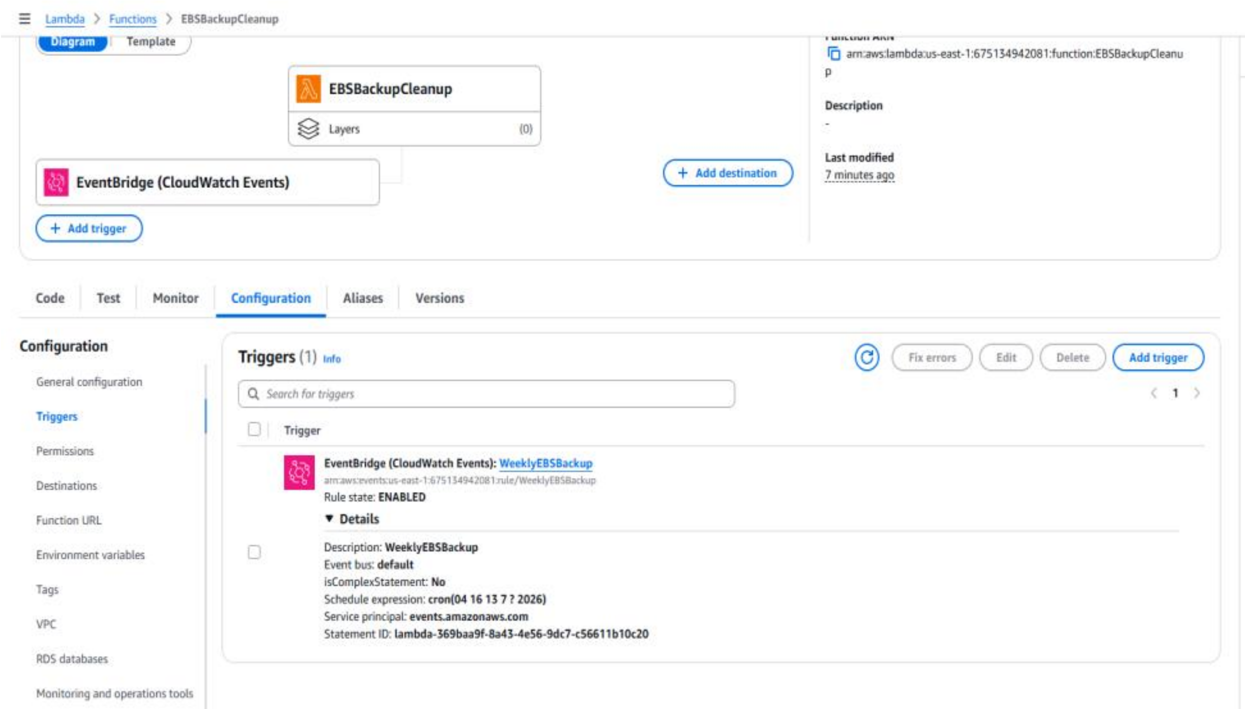
1. Search bar → **EventBridge** → open the EventBridge console.
2. Left sidebar → **Rules** → **Create rule**.
3. **Name**: WeeklyEBSBackup
4. **Rule type**: Schedule → **Next**.
5. **Schedule pattern**: select **A fine-grained schedule that runs at a specific time** → choose **Cron-based schedule**.
6. Enter cron expression: 0 3 ? * SUN * (runs every Sunday at 03:00 UTC) → **Next**.
7. **Target type**: AWS service.
8. **Select a target**: Lambda function.

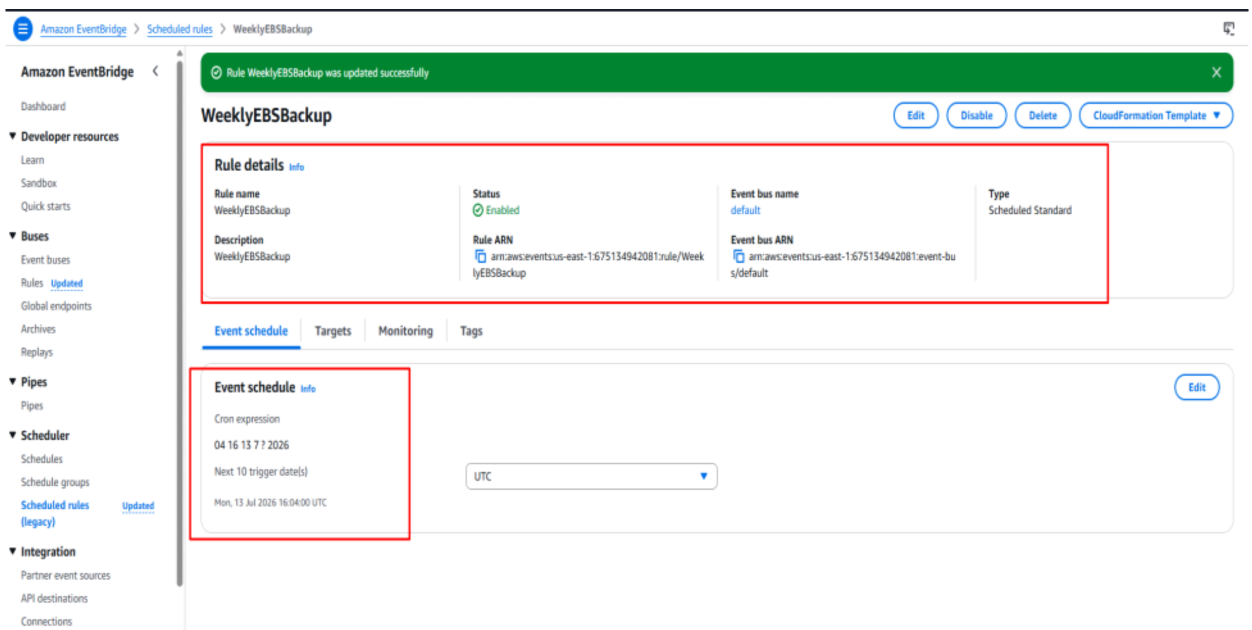
9. **Function**: select EBSBackupCleanup → **Next**.
10. Review and click **Create rule**.
    - EventBridge automatically adds the required invoke permission to the Lambda when created through the console.



cron(**minutes hours day-of-month month day-of-week year**)

Schedule expression: **cron(04 16 13 7 ? 2026)**

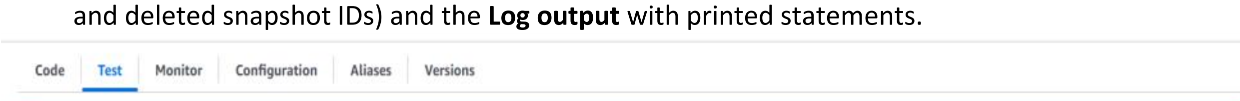


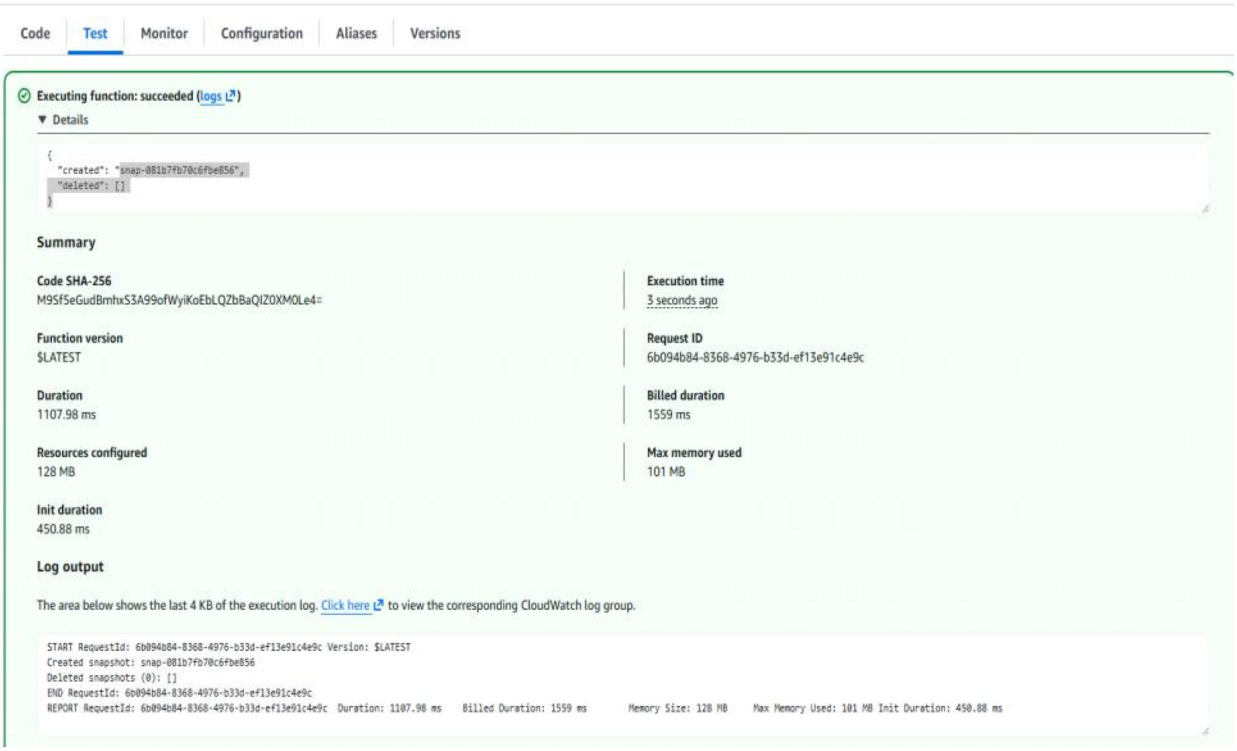


5. Testing

Console Navigation — Manual Trigger

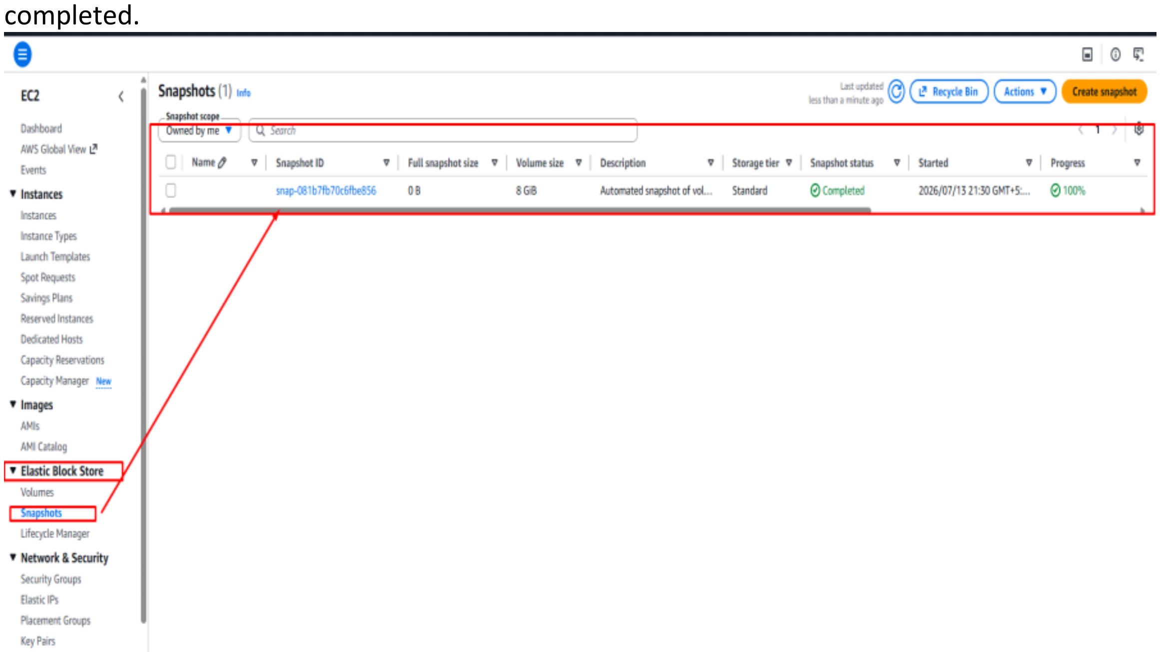
1. Open the **Lambda** console → select EBSBackupCleanup.
2. Click the **Test** tab.
3. **Event name**: manual-test
4. Leave the default JSON {} (no input needed) → **Save**.
5. Click **Test**.
6. Check the **Execution results** panel at the top — expand **Details** to see the returned JSON (created and deleted snapshot IDs) and the **Log output** with printed statements.



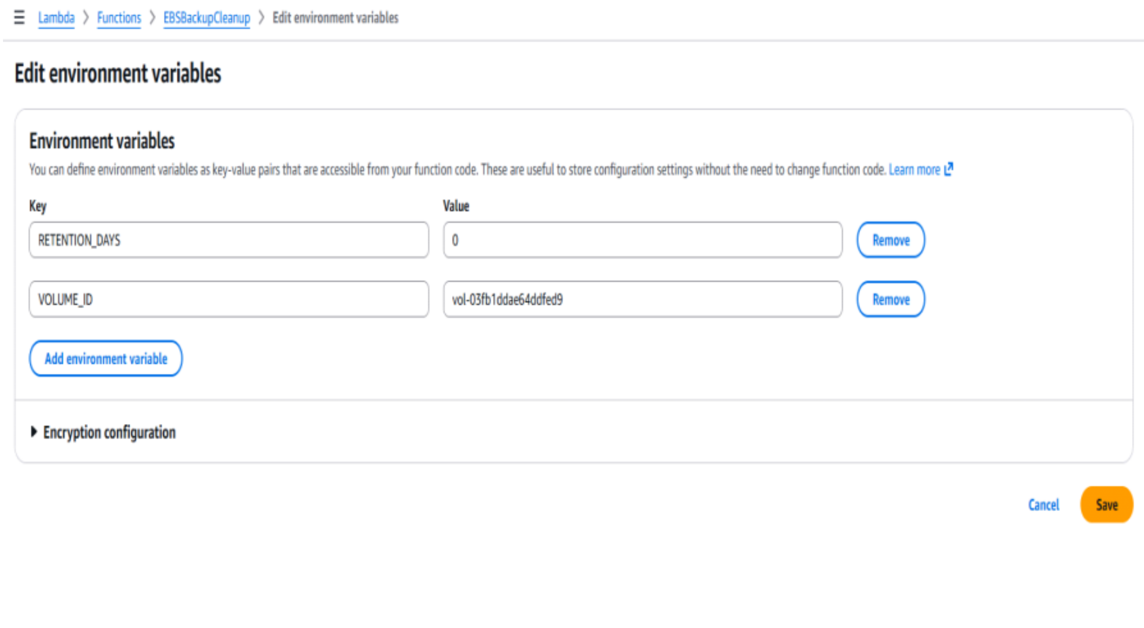


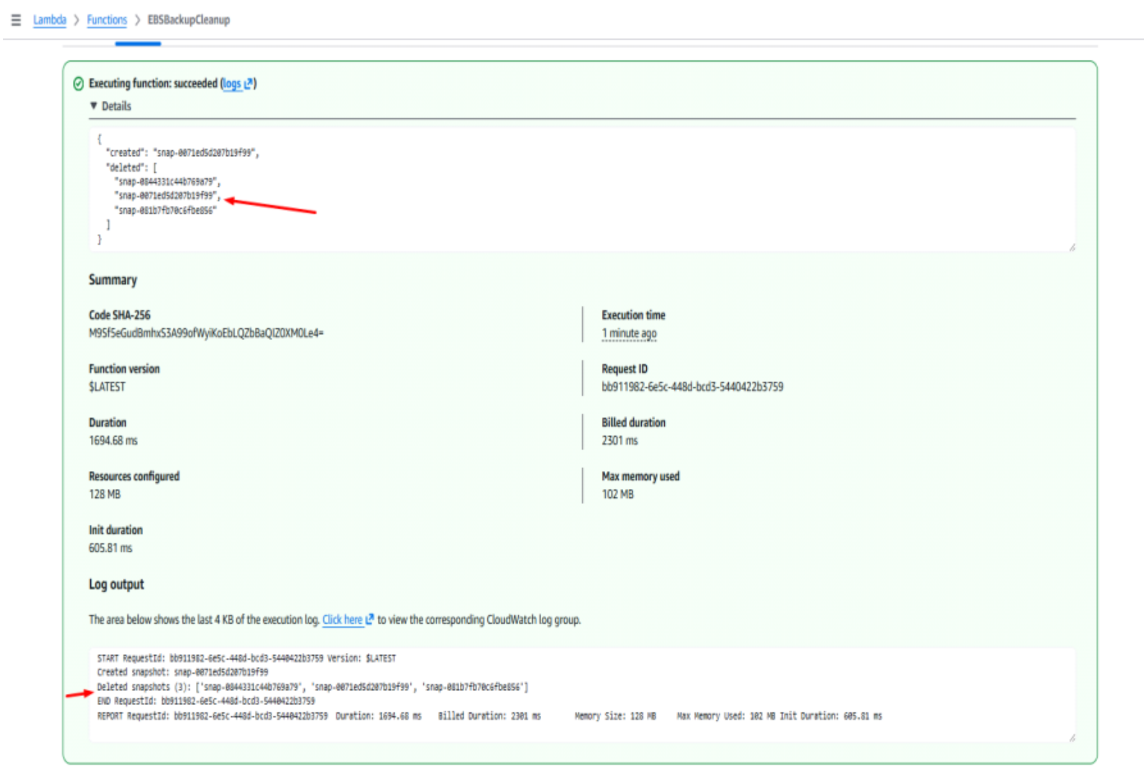
Console Navigation — Verify in EC2 Console

1. Go to **EC2** console → **Elastic Block Store** → **Snapshots** (left sidebar).
2. Use the search/filter bar: filter by tag CreatedBy = Lambda-Backup.
3. Confirm the new snapshot appears with **Started** timestamp matching your test run and status completed.

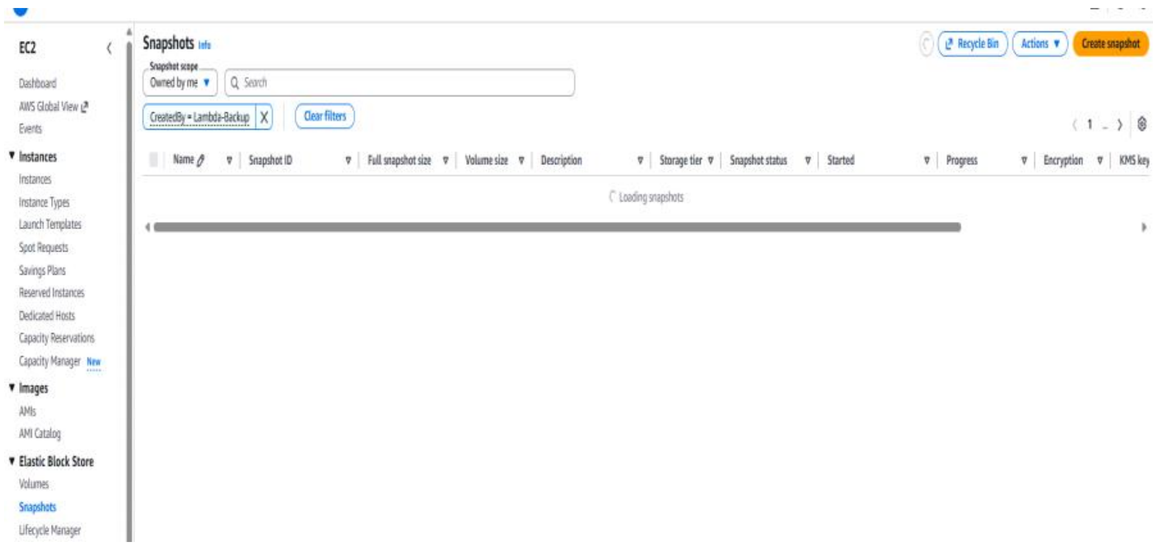


4. To test cleanup: go back to Lambda → **Configuration** → **Environment variables** → temporarily set RETENTION_DAYS to 0 → **Save** → re-run **Test** tab.

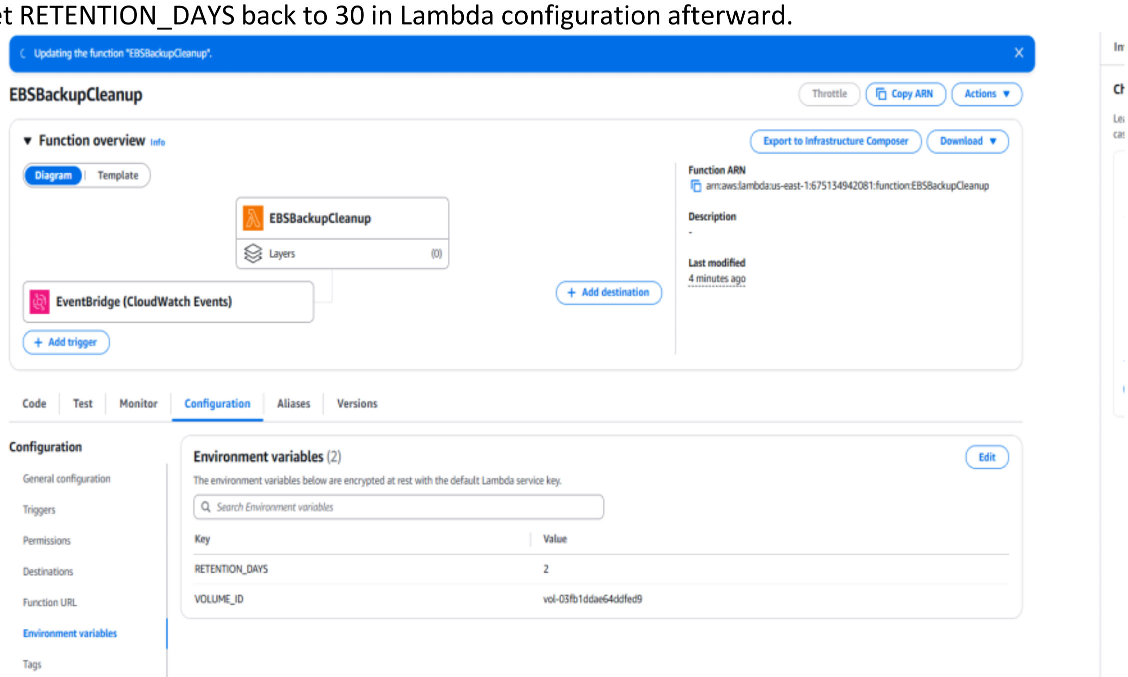


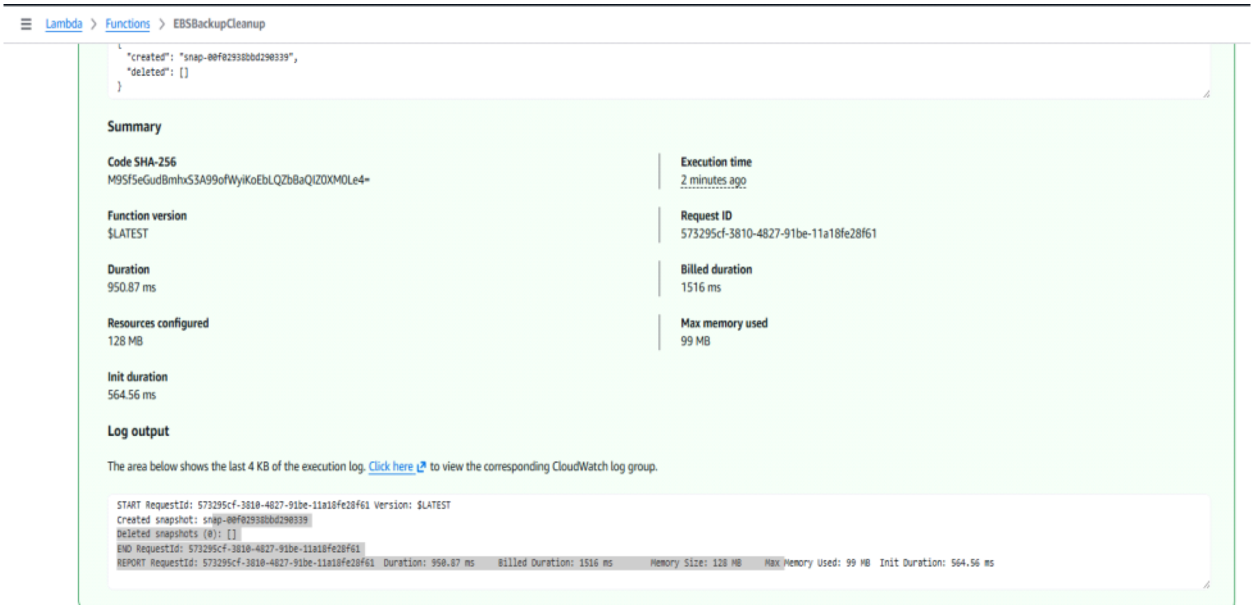


5. Refresh the Snapshots list in EC2 console — older tagged snapshots should disappear.

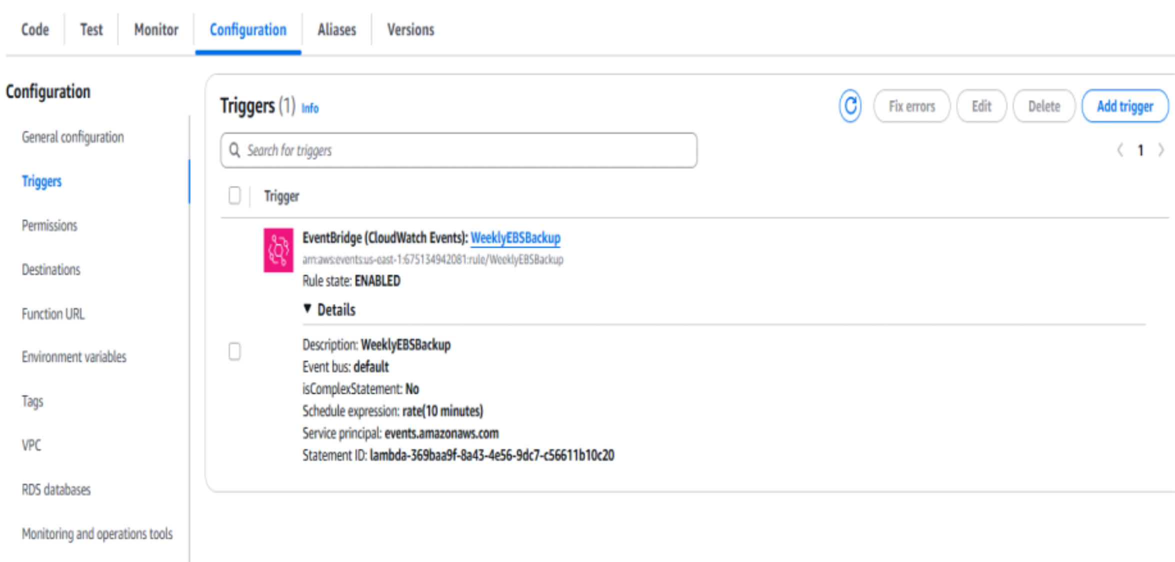


6. Reset RETENTION_DAYS back to 30 in Lambda configuration afterward.



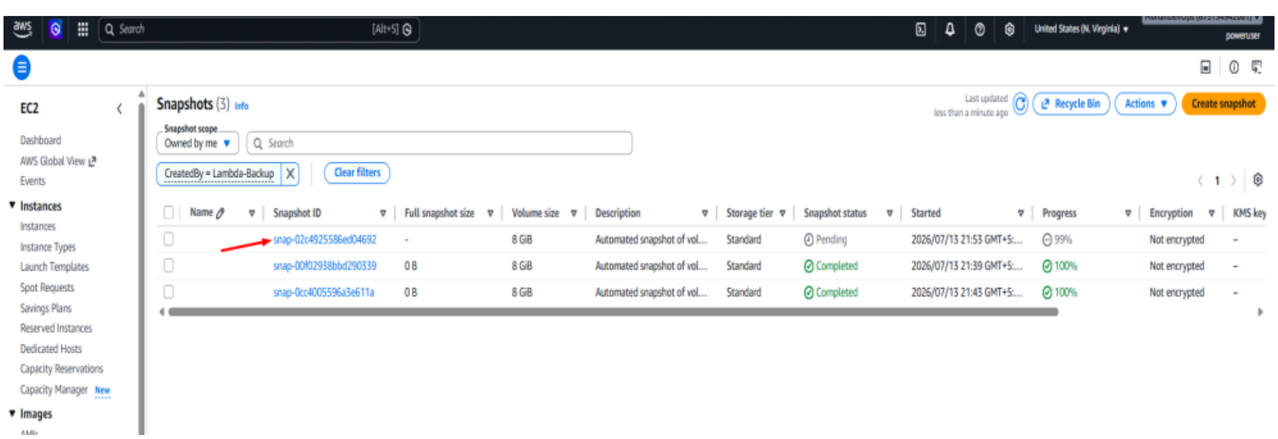


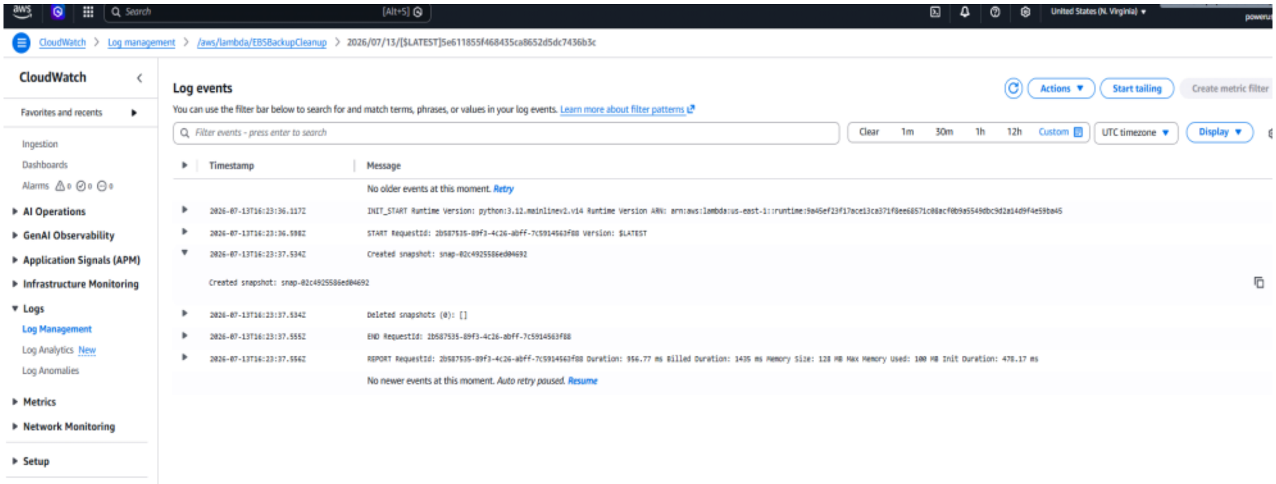
Change Cron run every 10 min for Testing



Console Navigation — View Logs

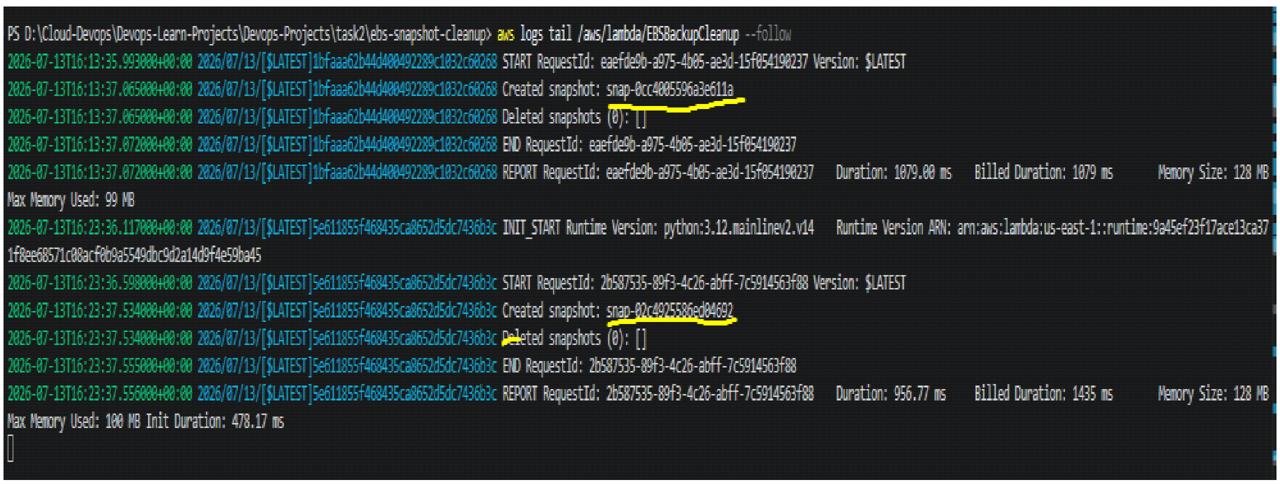
1. Lambda console → EBSBackupCleanup → **Monitor** tab → **View CloudWatch logs**.
2. Click the most recent **Log stream** to see the printed created/deleted snapshot IDs.



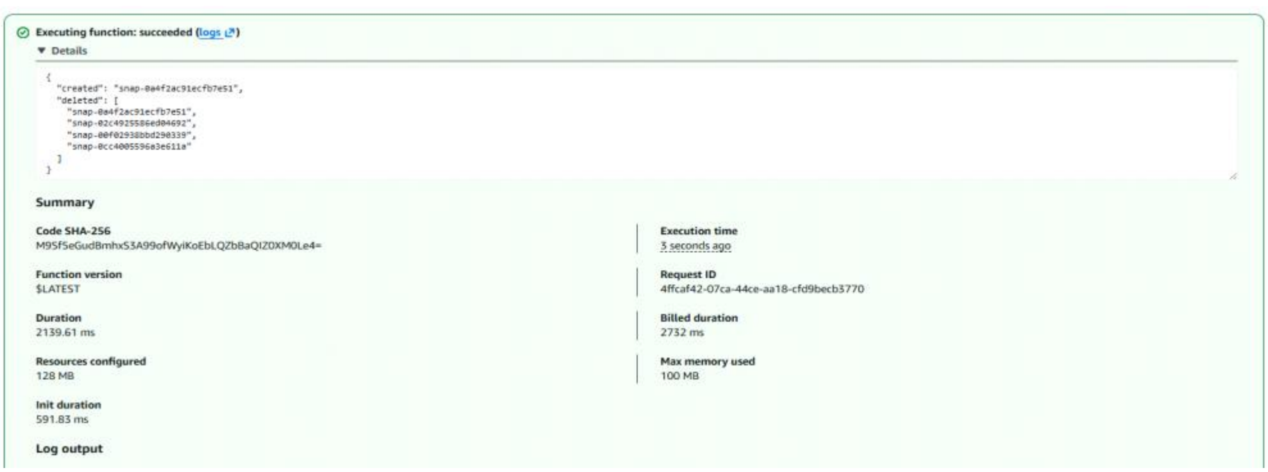


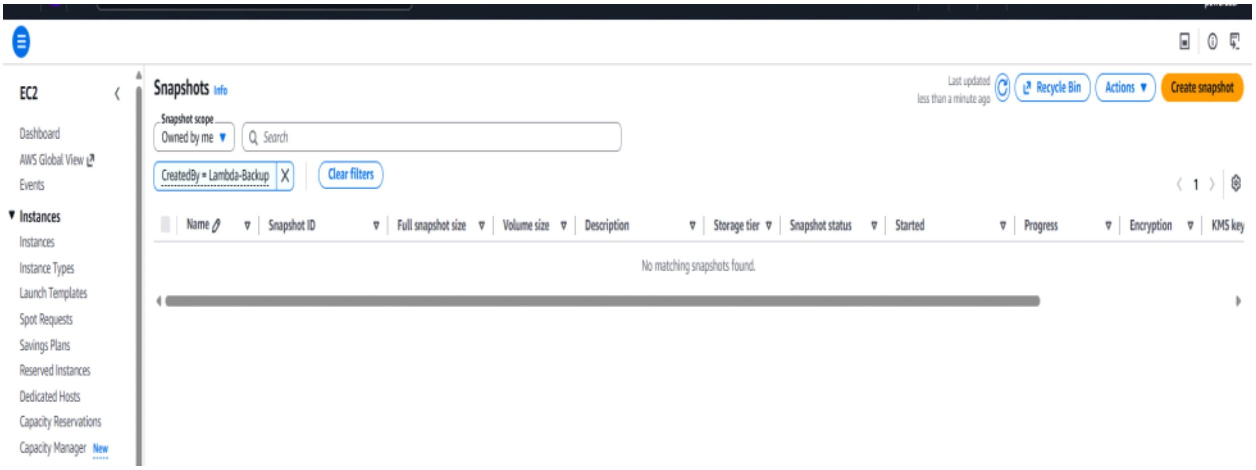
CLI Equivalent

aws logs tail /aws/lambda/EBSBackupCleanup --follow



Set to again >>> RETENTION_DAYS=0





6. Lambda vs. AWS Data Lifecycle Manager (DLM)

DLM Console Navigation (for comparison/reference):

1. EC2 console → left sidebar (bottom, under **Elastic Block Store**) → **Lifecycle Manager**.
2. **Create lifecycle policy** → choose **EBS snapshot policy**.
3. Select target volumes by tag, set schedule frequency and retention count/days, assign an IAM role (DLM has an AWS-managed service-linked role option).
4. No code required — this is DLM's core appeal for simple schedules.